

C Programming

10_struct_union

Попробуем создать кота..

struct (Структуры)

Структура — это объединение нескольких объектов, возможно, различного типа под одним именем, которое является типом структуры. В качестве объектов могут выступать переменные, массивы, указатели и другие структуры.

```
struct ИмяСтруктуры  
{  
    тип ИмяЭлемента1;  
    тип ИмяЭлемента2;  
    ...  
    тип ИмяЭлементan;  
};
```

```
struct ИмяСтруктуры имяПеременной;  
  
имяПеременной.ИмяЭлемента1 = Значение1;  
имяПеременной.ИмяЭлемента2 = Значение2;
```

```
struct Point { // Структура точка(x, y)
    int x;
    int y;
};
```

```
struct Line { // Структура прямая
    struct Point p1; // точка 1
    struct Point p2; // точка 2
};
```

```
struct Line my_line;
my_line.p1.x = -5;
my_line.p1.y = 4;
my_line.p2.x = 3;
my_line.p2.y = -7;
```

```
struct complex {  
    int re; // реальная часть  
    int im; // мнимая часть  
};
```

re		im				
		-5	6			
100	104	108				

Инициализация

```
// Определяем структуру Cat
struct cat {
    char *name;
    int age;
};
```

```
// Варианты инициализации структуры
struct cat tom = { "Tom", 5 };
struct cat felix = { .name = "Felix", .age = 4 };
struct cat mango;
mango.name = "Mango";
mango.age = 3;
```

Массив структур

```
struct funny {  
    char name[128];  
    char *fate;  
};
```

```
struct funny cats[] = {{ "Yaqub Qomarudin Dibizah", '👑' },  
                       { "Khalid Kashmiri", '🔪' },  
                       { "Khidir Karawitah", '🔪' },  
                       { "Ismail Ahmad Khan Nabawi", '🐎' },  
                       { "Utsman Abdul Jalil Shisha", '📄' },  
                       { "Muhammad Sumbul", '🔪' } };
```

```
printf("Cat name: %s", cats[1].name); // => "Khalid Kashmiri"
```

pointer to struct

```
struct test {  
    char a;  
    int b;  
};
```

```
struct test a; // переменная со структурой test  
struct test *ptr; // переменная-указатель на структуру  
test
```

```
ptr = &a; // берем адрес a и записываем его в ptr  
ptr->a = 'A'; // работаем с переменной a косвенно  
ptr->b = 555;
```

ptr->a тоже самое что и (*ptr).a. При обращении к элементу указателя на структуру используется '->', а при работе с обычной структурой '.'

sizeof struct

```
struct test {  
    int a;  
    int b;  
};
```

```
struct test t = { 'A', 49407 };  
sizeof(t) = sizeof(struct test) = ?
```

sizeof struct

```
struct test {  
    char a;  
    int b;  
};
```

```
struct test t = { 'A', 49407 };  
sizeof(t) = sizeof(struct test) = ?
```

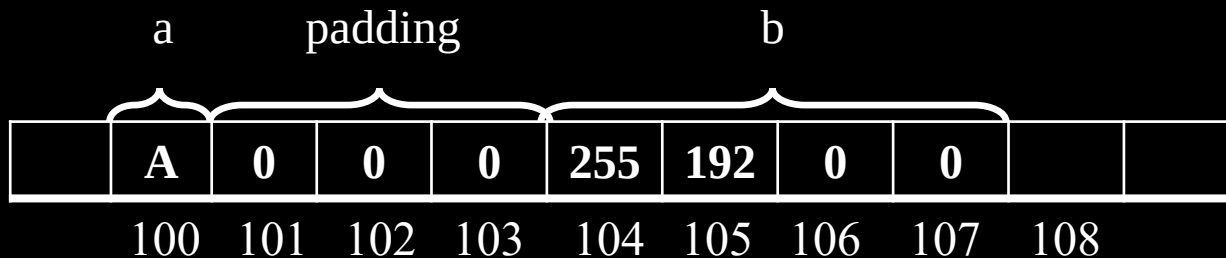
struct padding (Набивка/Добивание)

```
struct test {  
    char a;  
    int b;  
};
```

```
struct test t = { 'A', 49407 };
```

```
struct test {  
    char a;  
    int b;  
} __attribute__((packed));
```

```
struct test t;  
// sizeof(t) = 5
```



Компилятор возьмет в структуре самый большой размер переменной положит всю структуру начиная с адреса кратного этому размеру.

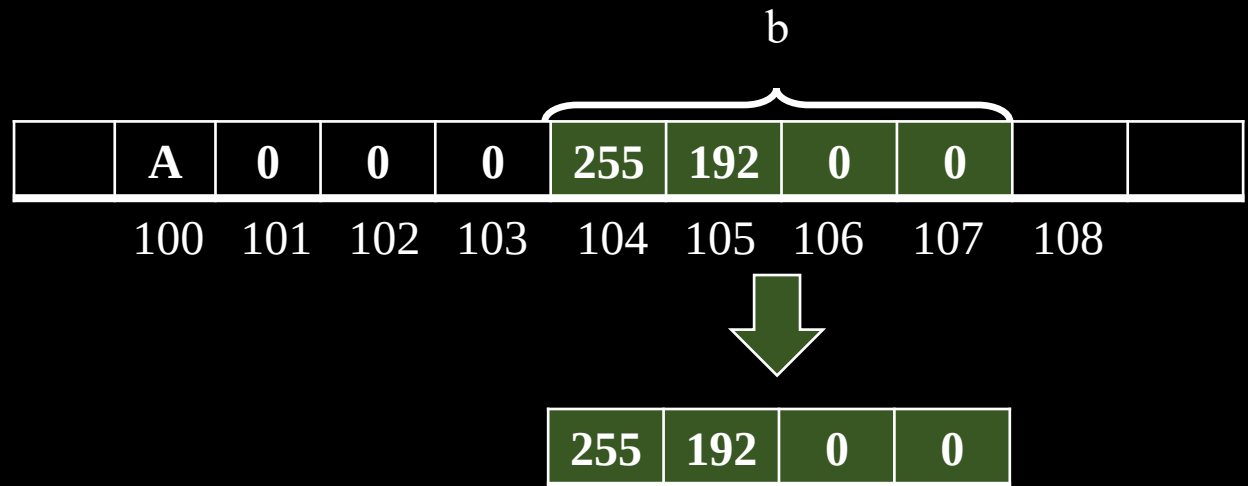
Alignment (Выравнивание)

Современные процессоры работают с числами, у которых размер кратен степени 2, т.к. у нас везде двоичная система исчисления. Процессор отлично работает с числами в 1 байт, 2, 4, 8, 16.. байт.

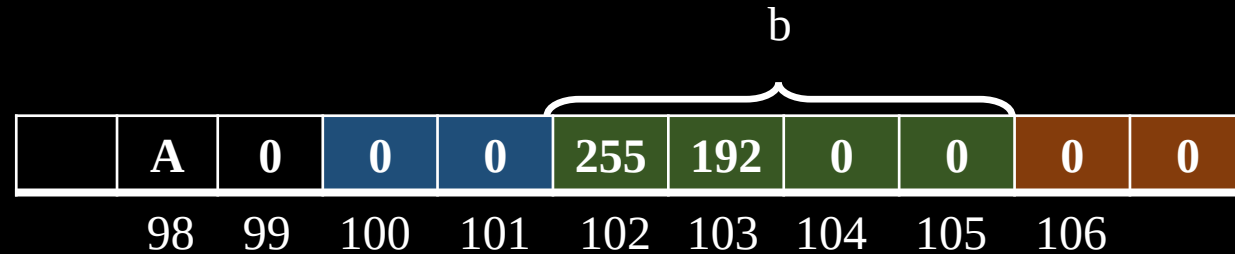
Процессор будет быстро работать с числами расположенными по адресу, который кратен их размеру. Если для 4 байтного числа адрес кратен 4, то процессор считает такое число за 1 такт.

Такой подход наиболее эффективен при проектировании процессоров и позволяет делать процессоры более компактными.

Выравнивает данные по адресам компилятор



Alignment (Выравнивание)



Если данные у нас не выравнены по адресу кратному 4 для числа размера 4 байт.

Процессор считывает два числа по кратным адресам 4 и затем сдвинет байты в нужные позиции и затем сложит. Итого доступ к числу b будет примерно 4 такта, т.е. в 4 раза дольше чем когда структура была выравнена.



```
struct test {  
    char a;  
    int b;  
    char c;  
    int d;  
};
```

`sizeof(struct test)` = ?

```
struct test {  
    int b;  
    int d;  
    char a;  
    char c;  
};
```

`sizeof(struct test)` = ?

Битовые поля структуры

```
struct byte {  
    unsigned char b0:1;  
    unsigned char b1:1;  
    unsigned char b2:1;  
    unsigned char b3:1;  
    unsigned char b4:1;  
    unsigned char b5:1;  
    unsigned char b6:1;  
    unsigned char b7:1;  
};
```

```
char a = 0b00010001; // 17  
struct byte *bits; // указатель на структуру  
bits = (struct byte *)&a;  
  
bits->b0 = 0; // 0 бит числа a
```


Битовые поля структуры

```
struct byte {  
    unsigned char b0:1;  
    unsigned char b1:1;  
    unsigned char b2:1;  
    unsigned char b3:1;  
    unsigned char b4:1;  
    unsigned char b5:1;  
    unsigned char b6:1;  
    unsigned char b7:1;  
};
```

Почему все поля в структуре имеют тип данных **unsigned** char?

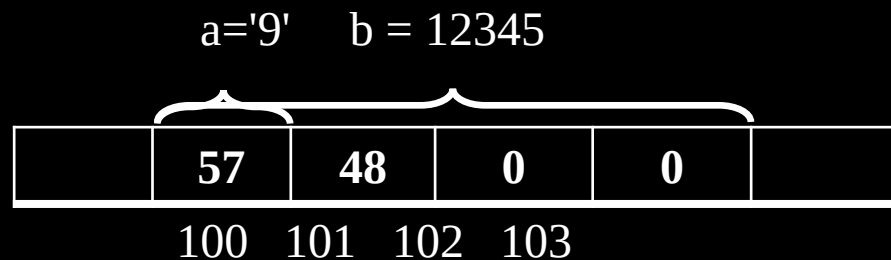
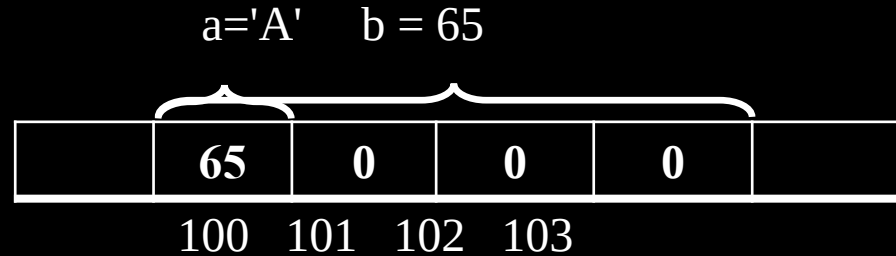
union (Объединения)

```
union my_union {  
    char a;  
    int b;  
};
```

`sizeof(union my_union) = 4`

```
union my_union test;  
test.a = 'A';
```

```
test.b = 12345;
```



union (Объединения)

```
struct test {  
    int type;  
    union u {  
        struct foo a;  
        struct bar b;  
    };  
};
```

```
union {  
    char a[4];  
    int b;  
} u;
```

enum (Перечисление)

```
enum colors {  
    RED, // по умолчанию будет = 0  
    GREEN, // = 1  
    BLUE = 5,  
    ORANGE // будет равно 6  
};  
...  
enum colors my_color = GREEN;  
  
switch (my_color) {  
    case RED:  
        // выполнить что-то если красный  
    case GREEN:  
        // выполнить что-то если зеленый  
}
```

Полезно когда хочется иметь строковый аналог числам. Например состояния программы:

1. работает
2. не работает
3. пауза

Вместо использования чисел более понятнее будет использовать слова.

typedef

```
typedef unsigned int uint;  
uint a;
```

```
typedef struct cats {  
    int age;  
    char name[64];  
} cats_t;  
cats_t cats = {3, "Mango"};
```

<stdint.h>

Github

<https://github.com/kruffka/C-Programming>

